



PRÁCTICA 1: DOCKER Y DOCKER COMPOSE

**ANGEL MAURICIO MOZO PÉREZ
EMMANUEL DAVID LANIS DACOSTA
JULIAN DAVID VEGA MARIN**

**UNIVERSIDAD AUTÓNOMA DE OCCIDENTE
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL
SANTIAGO DE CALI
2026**

Gestión de Usuarios con FastAPI, PostgreSQL, Prometheus y Grafana

Repositorio: <https://github.com/emmanueldavlenis-stack/Practica-Docker>

1. Explicación y justificación del proyecto

1.1 ¿Qué se construyó?

Una API de usuarios hecha con FastAPI y PostgreSQL que expone métricas de su comportamiento. Esas métricas las recoge Prometheus y se muestran en un dashboard de Grafana. Todo el sistema corre en contenedores Docker orquestados con Docker Compose.

```
POST /users → FastAPI → PostgreSQL
                ↓ /metrics
            Prometheus (consulta cada 5 s)
                ↓
            Grafana (dashboard)
```

Servicio	Imagen	Puerto	Redes
postgres	postgres:13	5432 (interno)	backend
fastapi	Construida con el Dockerfile del proyecto	8000	backend, analisis
prometheus	prom/prometheus	9090	analisis
grafana	grafana/grafana	3000	analisis

1.2 Primera parte: construcción manual

El objetivo fue hacer el proceso a mano: escribir el Dockerfile, construir la imagen y levantar cada contenedor con docker run. Más que aprender una herramienta, el propósito era reforzar lo visto en clase y entender cómo se pone a correr un contenedor, qué pasa por debajo y cómo se conectan las piezas entre sí.

1.3 Segunda parte: Docker Compose y buenas prácticas

Se usó Docker Compose para automatizar el mismo proceso, aplicando mejores prácticas: un solo archivo describe todo el sistema y un solo comando lo levanta (docker compose up -d --build). Las decisiones tomadas y su justificación son:

Decisión	Justificación
Redes (networks) en lugar de --link	--link está obsoleto y es difícil de escalar. Las redes definidas por el usuario resuelven los nombres de los servicios automáticamente y son la forma recomendada de comunicar contenedores.
Dos redes: backend y analisis	Aíslan la base de datos: Prometheus y Grafana no tienen acceso a PostgreSQL. Solo la API está en ambas redes y hace de puente.
healthcheck y depends_on con service_healthy	La API espera a que PostgreSQL esté realmente listo, no solo iniciado. Resuelve el error de conexión rechazada que ocurría en el arranque manual.
Volúmenes (pgdata, grafana_data)	Los datos de la base de datos y de Grafana sobreviven si se recrean los contenedores.

Decisión	Justificación
Nombre del servicio como host	La URL de la base de datos usa postgres y Prometheus apunta a fastapi:8000. Docker resuelve esos nombres, así que no se manejan IPs a mano.
Versiones fijas en requirements.txt	Hace la construcción reproducible: la imagen se comporta igual en cualquier máquina.

1.4 Qué aprendimos

- **Prometheus:** recolecta métricas de las aplicaciones y las guarda como series de tiempo. Funciona por pull: es Prometheus quien consulta el endpoint /metrics de la API cada cierto tiempo, y los servicios a consultar se indican en un archivo de configuración.
- **Grafana:** toma los datos de Prometheus y los presenta en dashboards con gráficas y paneles, lo que permite entender de un vistazo cómo se comporta una aplicación. Los dashboards se pueden exportar como JSON para versionarlos y reproducirlos.
- **Docker Compose:** permite describir en un solo archivo todos los servicios de una aplicación (imágenes, redes, volúmenes, variables de entorno y orden de arranque) y levantarlos con un solo comando. Es lo mismo que se hizo a mano con docker run, pero de forma declarativa, ordenada y reproducible.

2. Implementación con Docker y Docker Compose

2.1 Dockerfile de la API

```
FROM python:3.12-slim
WORKDIR /app
COPY ./app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- python:3.12-slim: imagen base ligera de Python.
- Se copia primero requirements.txt y se instalan las dependencias antes de copiar el código: así Docker reutiliza esa capa en caché mientras las dependencias no cambien y las reconstrucciones son más rápidas.
- --host 0.0.0.0: la API escucha en todas las interfaces del contenedor, necesario para poder publicarla en el puerto 8000.
- Dependencias fijadas: fastapi 0.115.0, uvicorn 0.30.6, sqlalchemy 2.0.35, psycopg2-binary 2.9.9 y prometheus-client 0.21.0.

2.2 docker-compose.yml

Servicio	Configuración principal
postgres	Imagen postgres:13; usuario, contraseña y base de datos por variables de entorno; volumen pgdata para persistir los datos; healthcheck con pg_isready cada 5 s (5 reintentos); red backend.

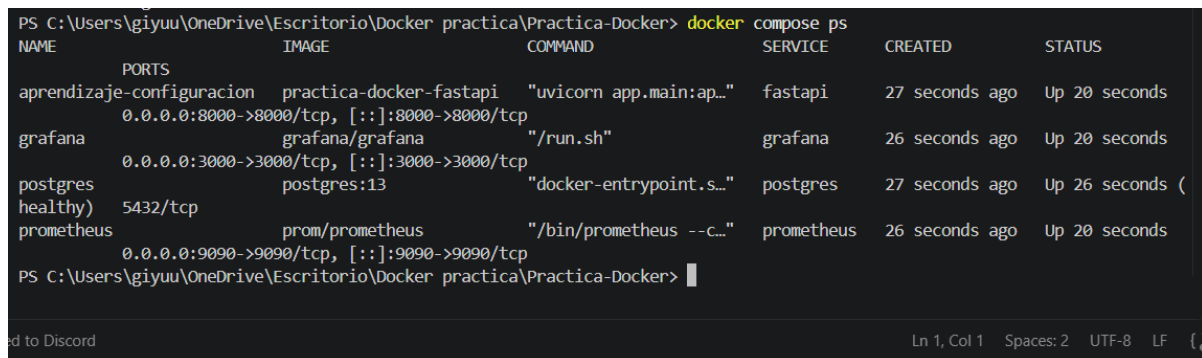
Servicio	Configuración principal
fastapi	Se construye con el Dockerfile del proyecto; restart: always; puerto 8000:8000; depends_on postgres con condition: service_healthy; redes backend y analisis.
prometheus	Imagen prom/prometheus; puerto 9090; monta prometheus/prometheus.yml; depende de fastapi; red analisis.
grafana	Imagen grafana/grafana; puerto 3000; usuario y contraseña de administración por variables de entorno; volumen grafana_data y carpetas de provisioning (fuente de datos y dashboards); depende de prometheus; red analisis.

Se declaran dos volúmenes (pgdata y grafana_data) y dos redes tipo bridge (backend y analisis).

2.3 Puesta en marcha

```
docker compose up -d --build
docker compose ps
```

Los cuatro servicios quedan en estado running y postgres en healthy (Figura 1). El contenedor de la API se llama aprendizaje-configuracion porque así se definió su container_name en el compose; el servicio sigue siendo fastapi. Para detener todo se usa docker compose down (conserva los datos) o docker compose down -v (borra también los volúmenes).



```
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
aprendizaje-configuracion          practica-docker-fastapi             "uvicorn app.main:ap..."           fastapi     27 seconds ago Up 20 seconds
grafana                            grafana/grafana                     "/run.sh"                             grafana     26 seconds ago Up 20 seconds
postgres                           postgres:13                          "docker-entrypoint.s..."           postgres    27 seconds ago Up 26 seconds (
healthy)
prometheus                         prom/prometheus                     "/bin/prometheus --c..."           prometheus  26 seconds ago Up 20 seconds
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker>
```

2.4 Pruebas de la API

Método	Ruta	Descripción
GET	/users	Lista todos los usuarios
POST	/users?name=...&age=...	Crea un usuario
GET	/metrics	Métricas en formato Prometheus
GET	/docs	Documentación interactiva (Swagger UI)

Desde PowerShell se usa curl.exe (curl es un alias de otro comando). La Figura 2 muestra la ejecución de seed.py, que crea 10 usuarios (respuesta 200 en cada uno), y la lista que devuelve GET /users:

```
curl.exe -X POST "http://localhost:8000/users?name=Alice&age=25"
curl.exe -X GET "http://localhost:8000/users"
python seed.py # crea 10 usuarios de prueba
```

```
Problems Output Debug Console Terminal Ports
healthy) 5432/tcp
prometheus prom/prometheus "/bin/prometheus --c..." prometheus 26 seconds ago Up 20 seconds
0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker> python seed.py
200 {'name': 'User1', 'age': 50, 'id': 1}
200 {'name': 'User2', 'age': 51, 'id': 2}
200 {'name': 'User3', 'age': 63, 'id': 3}
200 {'name': 'User4', 'age': 55, 'id': 4}
200 {'name': 'User5', 'age': 25, 'id': 5}
200 {'name': 'User6', 'age': 28, 'id': 6}
200 {'name': 'User7', 'age': 23, 'id': 7}
200 {'name': 'User8', 'age': 29, 'id': 8}
200 {'name': 'User9', 'age': 39, 'id': 9}
200 {'name': 'User10', 'age': 54, 'id': 10}
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker> curl.exe -X GET "http://localhost:8000/users"
[{"name":"User1","age":50,"id":1},{"name":"User2","age":51,"id":2},{"name":"User3","age":63,"id":3},{"name":"User4","age":55,"id":4},
{"name":"User5","age":25,"id":5}, {"name":"User6","age":28,"id":6}, {"name":"User7","age":23,"id":7}, {"name":"User8","age":29,"id":8}, {"name":"User9","age":39,"id":9}, {"name":"User10","age":54,"id":10}]
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker>
Ln 1, Col 1 Spaces: 2 UTF-8 LF { }

PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker> curl.exe -X POST "http://localhost:8000/users?name=Alice&age=25"
{"name":"Alice","age":25,"id":11}
PS C:\Users\giyuu\OneDrive\Escritorio\Docker practica\Practica-Docker>
```

3. Dashboard de métricas en Grafana

3.1 Instrumentación de la API

La API expone el endpoint `/metrics` con una métrica propia, `user_created_total`, un contador que se incrementa cada vez que se crea un usuario, y con las métricas por defecto del proceso que publica la librería `prometheus-client`.

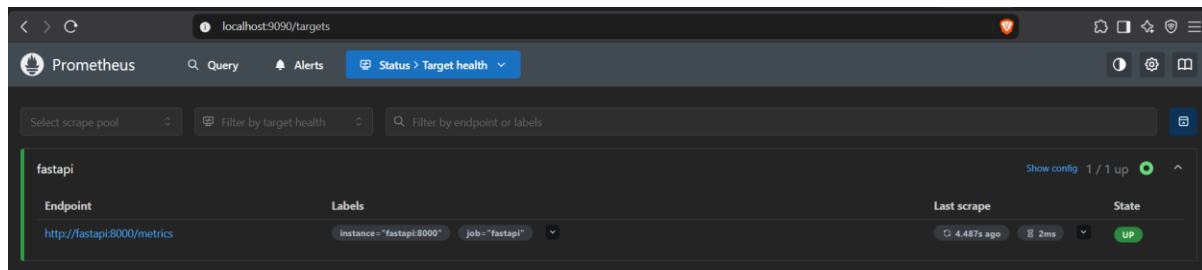
Métrica	Descripción
<code>user_created_total</code>	Usuarios creados desde que arrancó la app (contador propio)
<code>process_cpu_seconds_total</code>	CPU consumida por el proceso
<code>process_resident_memory_bytes</code>	Memoria RAM usada por la app
<code>process_open_fds</code>	Archivos y conexiones abiertos
<code>process_start_time_seconds</code>	Momento de arranque (permite calcular el tiempo de funcionamiento)
<code>up</code>	1 si Prometheus puede leer el target, 0 si no

Nota: `user_created_total` vive en memoria; si se reinicia el contenedor `fastapi` vuelve a 0, aunque los usuarios sigan en la base de datos.

3.2 Prometheus

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: "fastapi"
    static_configs:
      - targets: ["fastapi:8000"]
```

Prometheus consulta `http://fastapi:8000/metrics` cada 5 segundos (en Compose se usa el nombre del servicio como host). Su interfaz está en `http://localhost:9090` y el estado de los targets en `/targets`, donde `fastapi:8000` debe aparecer en UP.

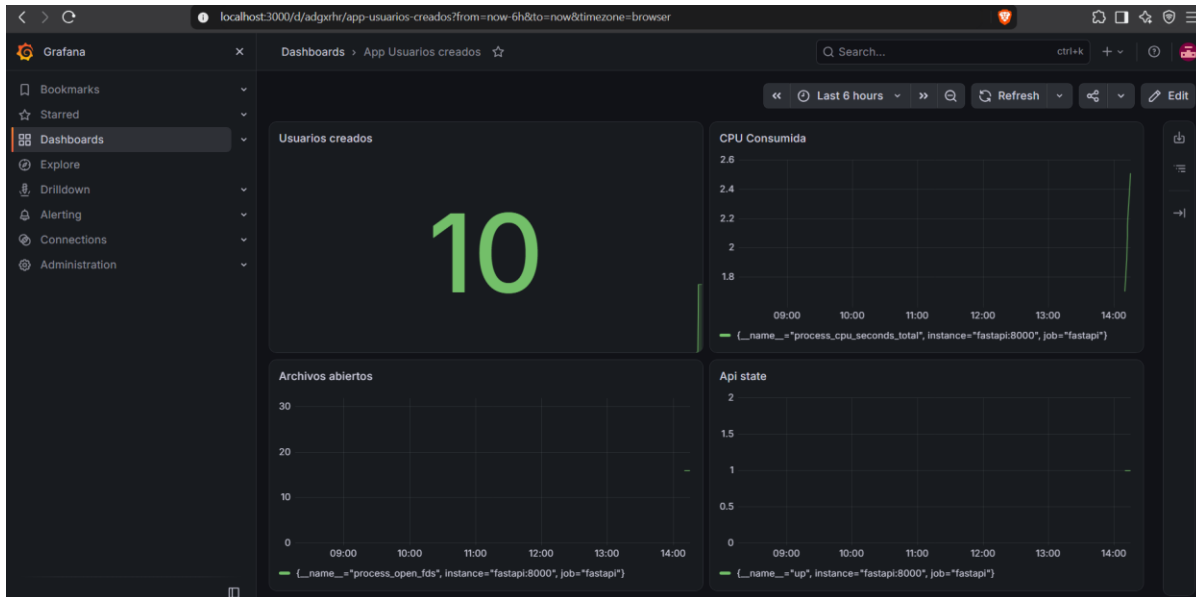


3.3 Grafana

Grafana (`http://localhost:3000`, usuario admin) se configura con provisioning: la fuente de datos Prometheus (`http://prometheus:9090`) se carga automáticamente al arrancar, definida en `grafana/provisioning/datasources/datasource.yml`. El dashboard se exportó como JSON (`grafana/Dashboard_Grafana.json`) y se puede importar desde Dashboards → New → Import → Upload dashboard JSON file.

El dashboard se llama "App Usuarios creados" y tiene cuatro paneles:

Panel	Tipo	Consulta (PromQL)	Qué muestra
Usuarios creados	Stat	<code>user_created_total</code>	Total de usuarios creados desde el arranque de la API
CPU Consumida	Serie de tiempo	<code>process_cpu_seconds_total</code>	CPU acumulada por el proceso de la API
Archivos abiertos	Serie de tiempo	<code>process_open_fds</code>	Archivos y conexiones abiertos por la API
Api state	Serie de tiempo	<code>up</code>	Disponibilidad de la API (1 = en línea, 0 = caída)



4. Repositorio y cumplimiento de la rúbrica

Todo el código y la documentación están versionados en el repositorio de GitHub:

```
docker-compose.yml
Dockerfile
seed.py
README.md
reflexion.md
app/          (main.py, requirements.txt, app/database.py, app/models.py)
grafana/      (Dashboard_Grafana.json y provisioning/)
prometheus/   (prometheus.yml)
```

Criterio de evaluación	Puntos	Dónde se cumple
Explicación y justificación del proyecto	1	Sección 1 de este documento y reflexion.md del repositorio
Dashboard con métricas en Grafana	2	Sección 3 y grafana/Dashboard_Grafana.json
Implementación completa con Docker y Docker Compose	2	Sección 2, Dockerfile y docker-compose.yml
Código y documentación en un repositorio Git	—	Repositorio de GitHub (sección 4)

5. Conclusión

Hacer primero el proceso manual permitió entender qué automatiza Docker Compose y por qué: redes, volúmenes, variables de entorno y orden de arranque son piezas que se configuran a mano en docker run, y Compose las reúne en un archivo declarativo y reproducible. Añadir Prometheus y Grafana mostró que desplegar una aplicación no termina cuando arranca: también hay que poder observar su comportamiento. El resultado es un sistema completo que se levanta con un solo comando y cuyo funcionamiento se puede medir y visualizar.